

Utiliser et modifier un notebook sur Google Colab

Mode d'emploi = comment exécuter un notebook dans le cloud, sans rien installer sur votre PC local.

On prend ici l'exemple de « Python in the Physical Chemistry Lab », alias PPCL.ipynb. La plupart des instructions et commentaires sont transférables à tout notebook.

Google Colab permet d'exécuter ce notebook directement dans votre navigateur, sur les serveurs de Google, sans installer Python ni aucune bibliothèque sur votre ordinateur. C'est une alternative pratique à une installation locale de JupyterLab, en particulier sur une machine où vous ne pouvez rien installer. Il vous suffit d'un compte Google.

Un notebook Python peut être vu, en première approche, comme une super calculatrice graphique : on y tape des expressions, on obtient des résultats immédiats, et on peut enchaîner et réutiliser les calculs à volonté.

Objectif du notebook PPCL... comme de tout autre notebook servant de support numérique à des cours/TD. Il ne s'agit pas de devenir un pro de l'algorithmique et de Python. L'objectif est de savoir **écrire du code de base**, et surtout de savoir **lire, interpréter, analyser et corriger** un code fourni par ailleurs, par exemple par une **IA générative**. À l'heure où l'on obtient du code en quelques secondes, l'essentiel n'est plus de le produire, mais de **comprendre ce qu'il fait et de juger s'il est correct**. La plus-value du support PPCL, par rapport à d'autres méthodes d'apprentissage de Python, est justement que tout se fait **dans un contexte chimique** : chaque notion est le plus souvent mise en pratique sur de vrais problèmes de chimie physique.

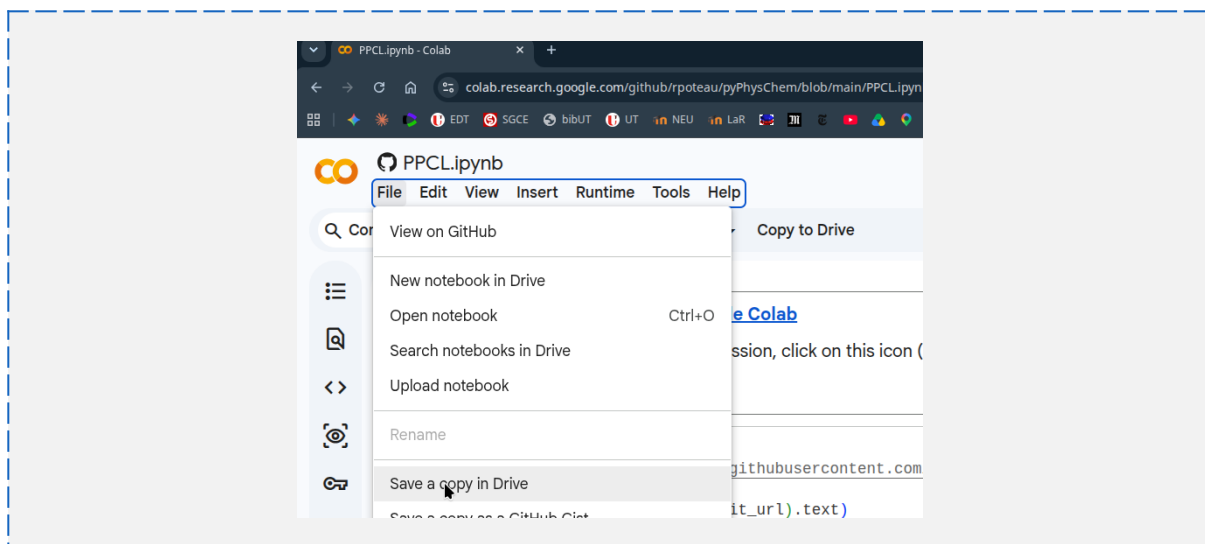
Enfin, les notebooks mis à votre disposition sont **souvent rédigés en anglais** : c'est une excellente occasion, pour des francophones, de se familiariser avec **l'anglais scientifique**, langue internationale incontournable dans un contexte recherche/développement/innovation.

1. La première fois : ouvrir et s'appropriier le notebook

1. Rendez-vous sur la page du notebook, par exemple : github.com/rpoteau/pyPhysChem/blob/main/PPCL.ipynb (remplacer par le lien de votre notebook de travail)
2. Cliquez sur le badge « **Open in Colab** » en haut du notebook. Connectez-vous avec votre compte Google si ce n'est pas déjà fait.



3. Le notebook s'ouvre dans Colab, mais en **lecture seule** (il vient de GitHub). Pour pouvoir sauvegarder votre travail, allez dans File > Save a copy in Drive.

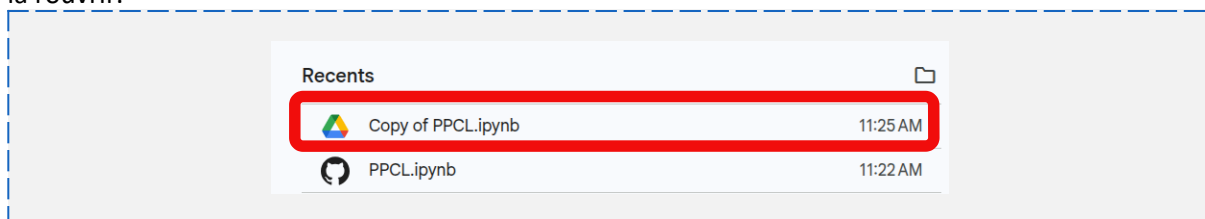


4. Désormais, vous travaillez sur **cette copie**, enregistrée dans votre espace **Google Drive**. C'est elle qu'il faudra rouvrir à chaque séance.

À retenir : cette étape ne se fait qu'**une seule fois**. Ensuite, votre copie personnelle vit dans votre Drive, indépendante de la version GitHub.

2. Reprendre le travail lors d'une séance suivante

1. Allez directement sur colab.research.google.com et connectez-vous.
2. Votre **copie de PPCL** apparaît **en tête des fichiers récents**, sur la page d'accueil. Cliquez dessus pour la rouvrir.



Important : à chaque réouverture, vous devez **ré-exécuter les deux premières cellules d'initialisation** (voir §5). Colab efface l'environnement entre deux sessions.

3. Comprendre le document

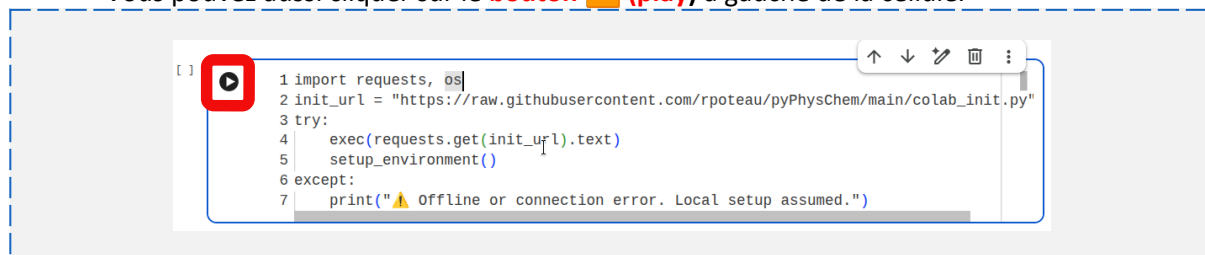
Ce document est un **notebook** : il mêle dans une même page du **texte**, des **images**, des **équations** et du **code** que vous pouvez exécuter. Il est composé de **cellules**, de deux types :

- **Cellules de code** : pour écrire et exécuter du Python. Le résultat s'affiche juste en dessous.
- **Cellules de texte (Markdown)** : pour les explications, titres, figures et équations. Si vous double-cliquez dessus par erreur, vous verrez le code de mise en forme : ré-exécutez la cellule pour retrouver l'affichage normal.

4. Exécuter les cellules

Dans Colab, **rien ne s'exécute automatiquement** : écrire $a = 2$ dans une cellule ne crée pas la variable pour autant : tant que vous n'avez pas exécuté la cellule, a n'existe pas. C'est à **vous de lancer chaque cellule**.

- Cliquez dans une cellule, puis appuyez sur **Maj + Entrée** (Shift + Enter) : la cellule s'exécute et le curseur passe à la suivante.
- Autre raccourci : **Ctrl + Entrée** exécute la cellule **sans** passer à la suivante.
- Vous pouvez aussi cliquer sur le **bouton ▶ (play)** à gauche de la cellule.



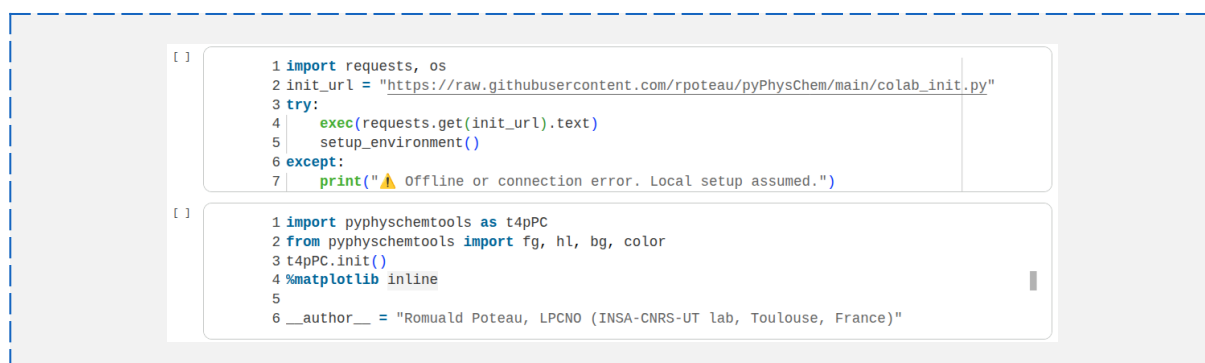
- Pendant l'exécution, le bouton play s'anime. Quand c'est terminé, les crochets à gauche [] deviennent un [n] (un **numéro**) indiquant l'ordre dans lequel les cellules ont été exécutées ; un [] vide signifie qu'elle n'a pas encore été exécutée.
- Vous pouvez **continuer à modifier d'autres cellules** pendant qu'une cellule tourne. En revanche, **une seule cellule s'exécute à la fois** : si vous en lancez d'autres, elles sont mises **en file d'attente** et s'exécuteront l'une après l'autre.
- Pour tout relancer proprement depuis le début : menu Runtime > Run all.

Bon réflexe : l'ordre d'exécution compte. Un notebook garde en mémoire ce que les cellules **déjà exécutées** ont produit. Si un résultat semble incohérent, réexécutez depuis le haut (ou Runtime > Restart session and run all).

5. Initialiser l'environnement (à faire à chaque session)

Le noyau (kernel). Les cellules de code sont exécutées par un **noyau Python** qui tourne en arrière-plan et **garde en mémoire toutes les variables** que vous avez créées. C'est pour cela qu'une variable définie dans une cellule reste disponible dans les suivantes—à condition de les avoir exécutées. Redémarrer le noyau (Runtime > Restart session) vide cette mémoire : il faut alors tout ré-exécuter depuis le début.

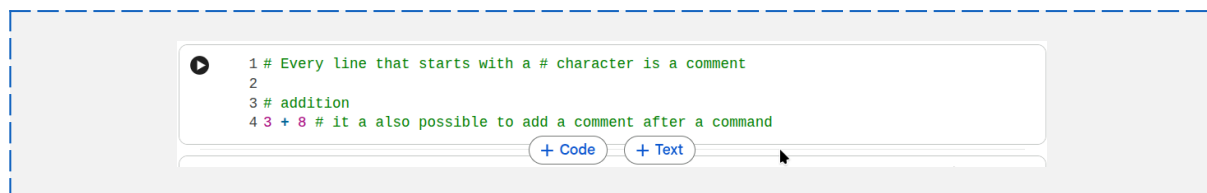
Une fois le notebook ouvert, vous devez préparer l'environnement Python et pyPhysChem en exécutant **les deux premières cellules de code, dans l'ordre**. La première **installe les bibliothèques et télécharge les fichiers d'accompagnement** : elle prend **environ 2 minutes**. Attendez qu'elle soit terminée avant de continuer.



Pourquoi refaire cela à chaque fois que le notebook est rouvert ? Colab tourne sur une **machine virtuelle temporaire** : après une période d'inactivité ou à la fermeture, elle est réinitialisée. Les bibliothèques installées et les fichiers téléchargés sont alors perdus ; mais votre notebook (code et résultats) reste sauvegardé dans votre Drive.

6. Insérer une nouvelle cellule (texte ou code)

Vous pouvez ajouter vos propres cellules, par exemple pour prendre des notes ou tester du code. Survolez l'espace entre deux cellules (ou le bas d'une cellule) : deux boutons apparaissent, « + Code » et « + Texte ». Cliquez sur celui voulu pour insérer une cellule à cet endroit.



- **+ Code** insère une cellule de code, exécutable avec Maj + Entrée.
- **+ Texte** insère une cellule de texte (Markdown) : tapez votre texte, puis Maj + Entrée pour l'afficher mis en forme.

7. Applications et exercices

Certains notebooks contiennent des exercices et applications, avec parfois des corrections intégrées. C'est le cas de PPCL.ipynb.

Les applications

Le notebook PPCL contient des **applications** : des exemples de code parfois **assez élaborés**. On ne vous demande **pas** de savoir les écrire vous-même. Leur objectif est de vous faire **comprendre ce que fait le code**, d'en saisir la logique et les différentes parties, et surtout de **prendre du recul** pour être capable de **transposer ces idées à d'autres problèmes**.

Les exercices

Le notebook peuvent aussi contenir des **exercices** sans solution proposée. Ils sont alors suivis d'une **cellule de code vide** dans laquelle écrire votre réponse. Ceux-ci sont à faire **par vous-même** : autant que possible, **évitons de recourir à une IA générative** — c'est en cherchant que vous acquerrez les réflexes de base.

La **solution est systématiquement fournie juste après**, mais masquée. La cellule de solution contient une ligne du type `%load ./SolutionsToExercices/...`. Pour la révéler :

- enlevez le commentaire (`#` suivi de l'espace) de la ligne `%load` (**raccourci clavier utile : Ctrl + /**) ;
- **exécutez la cellule une première fois** (Maj + Entrée) : cela charge le corrigé dans la cellule et remet le commentaire au début de la ligne `%load ...` ;
- **exécutez-la une seconde fois** : cela fait tourner le code et affiche le résultat.

Le bon usage : essayez d'abord **sérieusement**, puis comparez avec la solution. Révéler la réponse trop tôt vous prive de l'essentiel — apprendre à écrire et à corriger le code vous-même.

8. Sauvegarder votre travail

Une fois votre copie créée dans le Drive (§1), enregistrez à tout moment avec **Ctrl + S** ou **File > Save**. Votre code et les résultats affichés sont conservés. En revanche, le **contenu de la mémoire** (les variables) ne se sauvegarde pas : à la réouverture, réexécutez les cellules depuis le début.

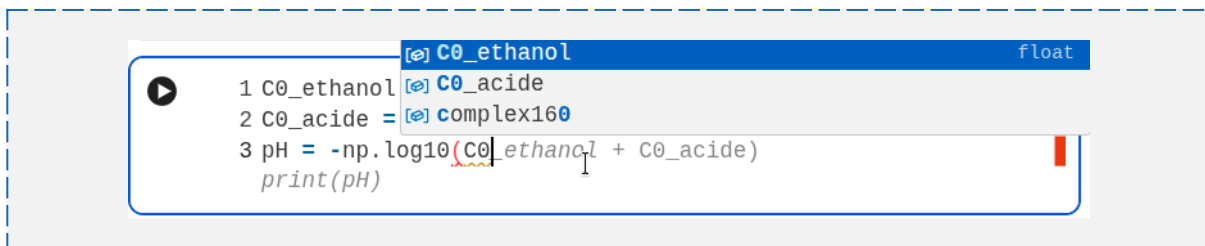
9. Erreurs fréquentes et bonnes habitudes

Python est **exigeant sur la forme** du code. Quelques erreurs reviennent souvent chez les débutants ; les connaître fait gagner beaucoup de temps.

- **L'indentation a un sens**. En Python, les espaces en début de ligne ne sont pas décoratifs : ils délimitent les blocs (à l'intérieur d'un `if`, d'une `for`, d'une fonction...). Une indentation incohérente

déclenche une **IndentationError**. Utilisez toujours le même pas (4 espaces) et ne mélangez pas espaces et tabulations.

- **Pas d'espace superflu.** Évitez un espace en trop en début de ligne, ou avant / après un opérateur de façon anarchique. Un espace involontaire devant une instruction (`a = 2`) peut suffire à provoquer une erreur.
- **Un code bien organisé.** Aérez : une instruction par ligne, des noms de variables parlants, et des commentaires (#) pour expliquer les étapes. Un code lisible est un code que vous (et une IA) pourrez relire et corriger facilement.
- **N'hésitez pas à donner des noms explicites, même longs.** Un nom comme `C0_ethanol` est bien plus clair que `c`. Pas d'inquiétude pour la frappe lorsque vous voudrez réutiliser la variable : comme la plupart des éditeurs, Google Colab propose l'**auto-complétion**. Commencez à taper le début d'un nom, puis appuyez sur Tab : Colab complète le nom, ou propose une liste de suggestions si plusieurs noms commencent pareil (naviguer avec les flèches ou bien avec la souris, puis valider avec Entrée).



```
1 C0_ethanol
2 C0_acide = complex160
3 pH = -np.log10(C0_ethanol + C0_acide)
print(pH)
```

- **Respecter la casse.** Python distingue majuscules et minuscules : Temperature et temperature sont deux variables différentes.
- **Exécuter dans l'ordre.** A variable or a library used before being defined / imported gives a NameError. Réexécutez depuis le haut en cas de doute.

Exemple : la même ligne, mal puis bien indentée.

```
x = 2      # espace superflu en début de ligne → IndentationError
x = 2      # correct
```

Lire le message d'erreur. Quand une cellule échoue, Python affiche un **message rouge** (un « traceback »). Lisez-le **de bas en haut** : la dernière ligne indique le type d'erreur (IndentationError, NameError...) et un court résumé. C'est votre première piste pour corriger.

Et l'IA générative dans tout ça ? Colab propose des fonctions d'IA (bouton « **Generate** » ✨, ou l'assistant Gemini) qui écrivent du code à partir d'une simple description. Au **début**, mieux vaut **s'en passer** : c'est en écrivant vous-même que vous acquerez les bases. Mais une fois que vous commencez à être à l'**aise en Python**, ces outils deviennent **très utiles et souvent très puissants**, à condition de toujours **relire, comprendre et vérifier** le code produit, comme le rappelle l'introduction de ce guide. Dans ces conditions, vous êtes fortement encouragés à vous en servir.

10. Se repérer : la table des matières



Pour naviguer facilement dans un notebook, Colab affiche une **table des matières** qui liste tous les titres de sections. Cliquez sur l'icône **≡ (Table des matières)** tout en haut de la **barre latérale gauche** : un panneau s'ouvre avec la liste des sections. Un clic sur un titre vous y amène directement.

Using and editing a notebook on Google Colab

User guide = how to run a notebook in the cloud, without installing anything on your local PC.

We use here the example of “Python in the Physical Chemistry Lab”, a.k.a. PPCL.ipynb. Most instructions and comments carry over to any notebook.

Google Colab lets you run this notebook directly in your browser, on Google's servers, without installing Python or any library on your computer. It is a convenient alternative to a local JupyterLab installation, especially on a machine where you cannot install anything. All you need is a Google account.

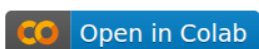
A Python notebook can be seen, as a first approximation, as a powerful graphing calculator: you type expressions, you get immediate results, and you can chain and reuse calculations at will.

*Purpose of the PPCL notebook... as with any other notebook used as digital support for courses/tutorials. The goal is **not** to become a pro at algorithmics and Python. The aim is to be able to **write basic code**, and above all to be able to **read, interpret, analyse and fix** code provided elsewhere, for example by a **generative AI**. At a time when code can be obtained in seconds, the essential is no longer to produce it, but to **understand what it does and judge whether it is correct**. The **added value of the PPCL support**, compared with other ways of learning Python, is precisely that everything is done **in a chemistry context**: each notion is most often put into practice on real physical-chemistry problems.*

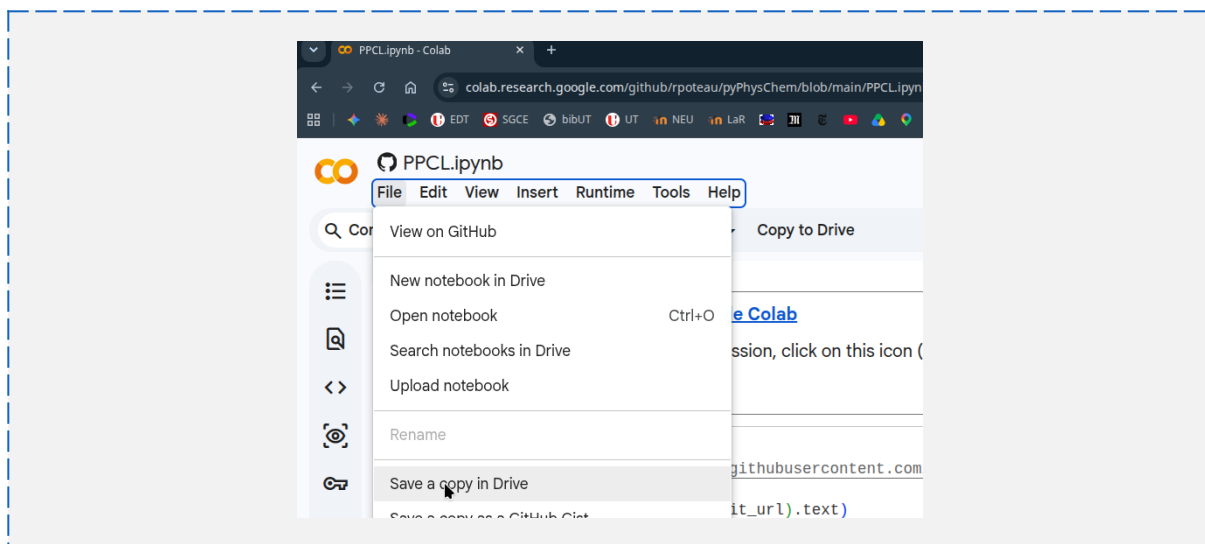
Enfin, les notebooks mis à votre disposition sont **souvent rédigés en anglais** : c'est une excellente occasion, pour des francophones, de se familiariser avec **l'anglais scientifique**, langue internationale incontournable dans un contexte recherche/développement/innovation.

1. The first time: open and make the notebook your own

- Go to the notebook page, for example: github.com/rpoteau/pyPhysChem/blob/main/PPCL.ipynb (replace with the link to your own working notebook)
- Click the “Open in Colab” badge at the top of the notebook. Sign in with your Google account if you have not already.



- The notebook opens in Colab, but read-only (it comes from GitHub). To be able to save your work, go to File > Save a copy in Drive.

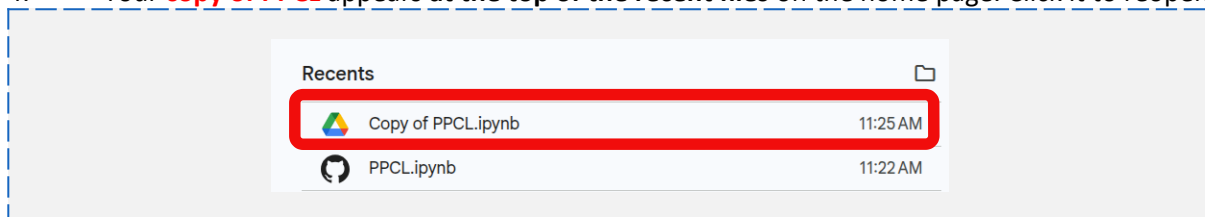


8. From now on, you work on this copy, saved in your Google Drive space. That is the one you will need to reopen at each session.

Key point: this step is done **only once**. After that, your personal copy lives in your Drive, independent of the GitHub version.

2. Resuming work in a later session

3. Go directly to colab.research.google.com and sign in.
4. Your **copy of PPCL** appears **at the top of the recent files** on the home page. Click it to reopen it.



Important: each time you reopen it, you must re-run the first two initialisation cells (see §5). Colab wipes the environment between sessions.

3. Understanding the document

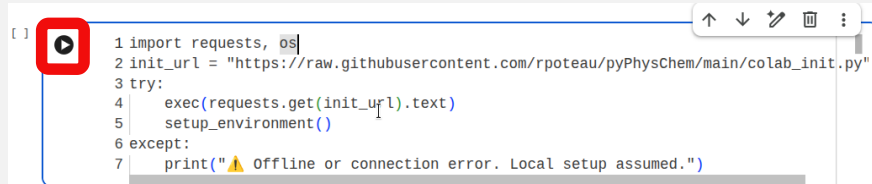
This document is a notebook: on a single page it mixes text, images, equations and code that you can run. It is made of cells, of two kinds:

- **Code cells:** to write and run Python. The result appears just below.
- **Text cells (Markdown):** for explanations, headings, figures and equations. If you double-click one by mistake, you will see the formatting source: re-run the cell to get the normal display back.

4. Running the cells

In Colab, nothing runs automatically: writing `a = 2` in a cell does not create the variable by itself — until you have run the cell, `a` does not exist. It is up to you to run each cell.

- Click in a cell, then press **Shift + Enter**: the cell runs and the cursor moves to the next one.
- Another shortcut: **Ctrl + Enter** runs the cell without moving to the next one.
- You can also click the **play button** to the left of the cell.



```
[ ] 1 import requests, os
2 init_url = "https://raw.githubusercontent.com/rpoteau/pyPhysChem/main/colab_init.py"
3 try:
4     exec(requests.get(init_url).text)
5     setup_environment()
6 except:
7     print("⚠️ Offline or connection error. Local setup assumed.")
```

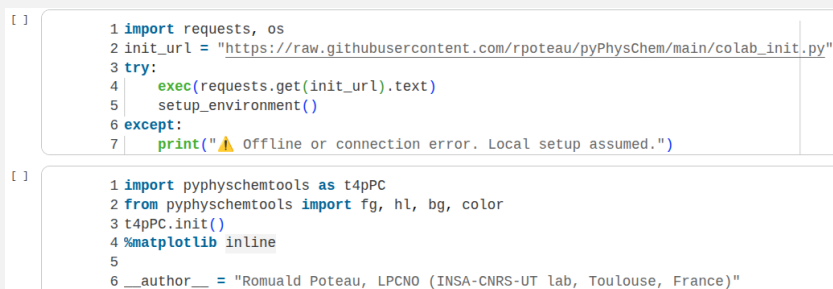
- While a cell runs, the play button animates. When it is done, the brackets on the left [] turn into a [n] (a number) indicating the order in which the cells were run; an empty [] means it has not been run yet.
- You can keep editing other cells while a long cell is running. However, only one cell runs at a time: if you launch others, they are queued and run one after another.
- To re-run everything cleanly from the start: menu Runtime > Run all.

Good habit: execution order matters. A notebook keeps in memory what the already-run cells produced. If a result looks inconsistent, re-run from the top (or Runtime > Restart session and run all).

5. Initialising the environment (do this every session)

The kernel. Code cells are run by a Python kernel that runs in the background and keeps in memory all the variables you have created. That is why a variable defined in one cell stays available in the following ones—provided you have run them. Restarting the kernel (Runtime > Restart session) clears this memory: you then have to re-run everything from the beginning.

Once the notebook is open, you must set up the Python and pyPhysChem environment by running the first two code cells, in order. The first one installs the libraries and downloads the companion files: it takes **about 2 minutes**. Wait until it has finished before continuing.



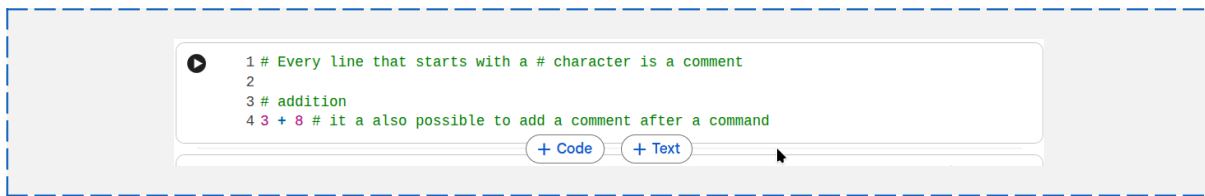
```
[ ] 1 import requests, os
2 init_url = "https://raw.githubusercontent.com/rpoteau/pyPhysChem/main/colab_init.py"
3 try:
4     exec(requests.get(init_url).text)
5     setup_environment()
6 except:
7     print("⚠️ Offline or connection error. Local setup assumed.")

[ ] 1 import pyphyschemtools as t4pPC
2 from pyphyschemtools import fg, hl, bg, color
3 t4pPC.init()
4 %matplotlib inline
5
6 __author__ = "Romuald Poteau, LPCNO (INSA-CNRS-UT lab, Toulouse, France)"
```

Why redo this every time the notebook is reopened? Colab runs on a **temporary virtual machine**: after a period of inactivity or on closing, it is reset. The installed libraries and downloaded files are then lost; but your notebook (code and results) stays saved in your Drive.

6. Inserting a new cell (text or code)

You can add your own cells, for example to take notes or test code. Hover over the space between two cells (or the bottom of a cell): two buttons appear, “+ Code” and “+ Text”. Click the one you want to insert a cell at that spot.



- **+ Code** inserts a code cell, runnable with Shift + Enter.
- **+ Text** inserts a text (Markdown) cell: type your text, then Shift + Enter to display it formatted.

7. Applications and exercises

Some notebooks contain exercises and applications, sometimes with built-in solutions. This is the case for *PPCL.ipynb*.

Applications

The **PPCL notebook** contains **applications**: code examples that are sometimes **fairly elaborate**. You are **not** expected to know how to write them yourself. Their purpose is to help you **understand what the code does**, grasp its logic and its different parts, and above all **step back** so you can **transpose these ideas to other problems**.

Exercises

The notebook may also contain exercises with no solution provided. They are then followed by an empty code cell in which to write your answer. These are to be done on your own: as much as possible, avoid resorting to a generative AI — it is by working things out that you build the basic reflexes.

The solution is always provided just after, but hidden. The solution cell contains a line such as `# %load ./SolutionsToExercises/....` To reveal it:

- **uncomment** (the # followed by the space) the **%load line** (handy keyboard shortcut: **Ctrl + /**);
- **run the cell a first time** (Shift + Enter): this loads the solution into the cell and puts the comment back at the start of the %load line;
- **run it a second time**: this executes the code and displays the result.

The right approach: try seriously first, then compare with the solution. Revealing the answer too early deprives you of the essential — learning to write and fix the code yourself.

8. Saving your work

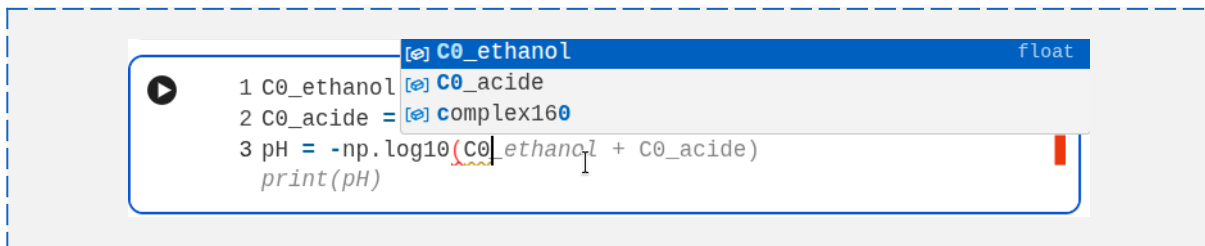
Once your copy is created in Drive (§1), save at any time with Ctrl + S or File > Save. Your code and the displayed results are kept. However, the contents of memory (the variables) are not saved: when you reopen, re-run the cells from the beginning.

9. Common errors and good habits

Python is demanding about the form of the code. A few errors come up often with beginners; knowing them saves a lot of time.

- **Indentation is meaningful.** In Python, leading spaces are not decorative: they delimit blocks (inside an if, a for, a function...). Inconsistent indentation triggers an **IndentationError**. Always use the same step (4 spaces) and do not mix spaces and tabs.
- **No stray spaces.** Avoid an extra space at the start of a line, or before / after an operator in a haphazard way. An unintended space in front of a statement (`a = 2`) can be enough to cause an error.
- **Well-organised code.** Give it room: one statement per line, meaningful variable names, and comments (#) to explain the steps. Readable code is code that you (and an AI) can re-read and fix easily.

- **Do not hesitate to use explicit names, even long ones.** A name like `C0_ethanol` is far clearer than `c`. No worries about typing when you want to reuse the variable: like most editors, Google Colab offers **auto-completion**. Start typing the beginning of a name, then press Tab: Colab completes the name, or offers a list of suggestions if several names start the same way (navigate with the arrows or the mouse, then confirm with Enter).



```

1 C0_ethanol
2 C0_acide = complex160
3 pH = -np.log10(C0_ethanol + C0_acide)
print(pH)

```

- **Respect case.** Python distinguishes uppercase and lowercase: `Temperature` et `temperature` are two different variables.
- **Run in order.** A variable or a library used before being defined / imported gives a `NameError`. Re-run from the top if in doubt.

Example: the same line, badly then correctly indented.

```

x = 2      # stray space at the start of the line → IndentationError
x = 2      # correct

```

Read the error message. When a cell fails, Python shows a red message (a “traceback”). Read it from bottom to top: the last line gives the error type (`IndentationError`, `NameError`...) and a short summary. That is your first lead for fixing it.

What about generative AI in all this? Colab offers AI features (the “**Generate**” ✨ button, or the Gemini assistant) that write code from a simple description. At first, it is better to **do without**: it is by writing yourself that you build the foundations. But once you start to feel **comfortable in Python**, these tools become **very useful and often very powerful**, provided you always **re-read, understand and check** the code produced, as the introduction to this guide recalls. Under those conditions, you are strongly encouraged to use them.

10. Finding your way: the table of contents



To move easily around a notebook, Colab shows a table of contents that lists all the section headings. Click the ≡ (Table of contents) icon at the very top of the left sidebar: a panel opens with the list of sections. Clicking a heading takes you straight there.